

Programming in the λ -Calculus From Church to Scott and back

Jan Martin Jansen

Faculty of Military Sciences,
Netherlands Defence Academy,
Den Helder, the Netherlands
`jm.jansen.04@nlda.nl`

Abstract. Although the λ -calculus is well known as a universal programming language, it is seldom used for actual programming or expressing algorithms. Here we demonstrate that it is possible to use the λ -calculus as a comprehensive formalism for programming by showing how to convert programs written in functional programming languages like **Clean** and **Haskell** to closed λ -expressions. The transformation is based on using the Scott-encoding for Algebraic Data Types instead of the more common Church encoding. In this way we not only obtain an encoding that is better comprehensible but that is also more efficient. As a proof of the pudding we provide an implementation of Eratosthenes' prime sieve algorithm as a self-contained, 143 character length, λ -expression.

1 The Church and Scott Encodings for Algebraic Data Types

The λ -calculus can be considered as the mother of all (functional) programming languages. Every course or textbook on λ -calculus (e.g. [1]) spends some time on showing how well-known programming constructs can be expressed in the λ -calculus. It commonly starts by explaining how to represent For natural numbers, in almost all cases the Church numerals are chosen as the leading example. The definition of Church numerals and operations on them shows that it is possible to use the λ -calculus for all kinds of computations and that it is indeed a universal programming language. The Church encoding can be generalized for the encoding of general Algebraic Data Types (see [2]). This encoding allows for a straightforward implementation of iterative (primitive recursive) or fold-like functions on data structures, but often requires complex and inefficient constructions for expressing general recursion.

It is less commonly known that there exist an alternative encoding of numbers and algebraic data structures in the λ -calculus. This encoding is relatively unknown, and independently (re)discovered by several authors (e.g. [9, 8, 10] and the author of this paper[6]), but originally attributed to Scott in an unpublished lecture which is cited in Curry, Hindley and Seldin ([4], page 504) as: *Dana Scott, A system of functional abstraction. Lectures delivered at University of California, Berkeley, Cal., 1962/63. Photocopy of a preliminary version, issued*

by *Stanford University, September 1963, furnished by author in 1968*.¹ We will therefore call it the *Scott* encoding. The encoding results in a representation that is very close to algebraic data types as they are used in most functional programming languages.

The goal of this paper is not to introduce a new (functional) programming language, but to show how the λ -calculus itself can be used as a concise programming formalism.

This paper starts with a discussion on Algebraic Data Types in Section 2. In Section 3 it discusses how the Scott and Church encoding can be used to encode Algebraic Data Types as λ -terms and how these approaches differ. Section 4 focusses on the encoding of recursive functions as λ -terms. In Section 5 the focus is on the conversion of a complete `Haskell` or `Clean` program to a single λ -term. The paper ends with a discussion in Section 7 and some conclusions in Section 8.

2 The Nature of Algebraic Data Types

Consider Algebraic Data Type (ADT) definitions in languages like `Clean` or `Haskell` such as tuples, booleans, temperature, maybe, natural (Peano) numbers and lists:

```
data Boolean    = True | False
data Tuple a b  = Tuple a b
data Temperature = Fahrenheit Int | Celsius Int
data Maybe a    = Nothing | Just a
data Nat        = Zero | Suc Nat
data List t     = Nil | Cons t (List t)
```

A type consists of one or more alternatives. Each alternative consist of a name, possibly followed by a number of arguments. Algebraic Data Types are used for several purposes:

- to make enumerations, like in `Boolean`;
- to package data, like in `Tuple`;
- to unite things of different kind in one type, like in `Maybe` and `Temperature`;
- to make recursive structures like in `Nat` and `List` (in fact to construct new types with an infinite number of elements).

The power of the ADT construction in modern functional programming languages is that one formalism can be used for all these purposes.

If we analyse the construction of ADT's more carefully, we see that constructor names are used for two purposes. First, they are used to distinguish the different cases in a single type definition (like `True` and `False` in `Boolean`, `Nothing` and `Just` in `Maybe` and `Fahrenheit` and `Celsius` in `Temperature`). Second, we need them for recognizing them as being part of a type and making type inferencing possible. Therefore, all constructor names must be different in a single functional

¹ I would like to thank Matthew Naylor for pointing me at this reference.

program (module). For distinguishing the different cases in a function definition, pattern matching on constructor names is used.

3 Representing Algebraic Data Types in the λ -calculus

In this section it is shown how to represent ADT's in the λ -calculus. First, we focus on non-recursive data types for which the Scott and Church encodings are the same and thereafter on recursive types for which the encodings differ.

3.1 Named λ -expressions

First, some remarks about the notation of λ -expressions. For convenience we will give λ -expressions sometimes names:

True $\equiv \lambda t f . t$

These names can be used as macro's in other λ -expressions. They are always written in italics:

True $(\lambda f g . f g) (\lambda f g . g f)$

is a short-hand for:

$(\lambda t f . t) (\lambda f g . f g) (\lambda f g . g f)$

Note that these macro names may not be used recursively, because this will lead to an infinite substitution process. Later on we discuss how to represent recursion in λ -expressions.

3.2 Expressing Enumeration Types in the λ -calculus

The simplest example of such a type is **Boolean**. We already noted that we use pattern matching for recognizing different cases (constructors). So we are actually looking for an alternative for pattern matching using λ -expressions. The simplest boolean pattern matching example is **if-then-else**:

ifte **True** $t f = t$
ifte **False** $t f = f$

But the same effect can easily be achieved by making **True** and **False** functions of two variables, selecting the first or second argument respectively and by making **ifte** the identity function. Therefore, the λ -calculus solution for this is straightforward:

True $\equiv \lambda t f . t$
False $\equiv \lambda t f . f$
ifte $\equiv \lambda i . i$

This is also the standard encoding used for booleans that can be found in λ -calculus courses and text books. Both Church and Scott use this encoding.

3.3 Expressing a Simple Container Type in the λ -calculus

`Tuple` is the simplest example of a container type. If we group data into a container, we also need constructions to get data out of it (projection functions). For `Tuple` this can be realized by pattern matching or by using the selection functions `fst` and `snd`. These functions are defined in Haskell as:

```
fst (Tuple a b) = a
snd (Tuple a b) = b
```

Containers can be expressed in the λ -calculus by using closures (partial applications). For `Tuple` the standard way to do this is:

```
Tuple  $\equiv \lambda a\ b\ f . f\ a\ b$ 
```

A tuple is a function that takes 3 arguments. If we supply only two, we have a closure. This closure can take a third argument, which should be a 2 argument function. This function is then applied to the first two arguments. The third argument is therefore called a continuation (the function with which the computation continues). It is now easy to find out what the definitions of `fst` and `snd` should be:

```
fst  $\equiv \lambda t . t\ (\lambda a\ b . a)$ 
snd  $\equiv \lambda t . t\ (\lambda a\ b . b)$ 
```

If applied to a tuple, they apply the tuple to a two argument function, that selects either the first (`fst`) or second (`snd`) argument.

Again, this definition of tuples is the standard one that can be found in λ -calculus text books and courses. Also for this case the Church and Scott encoding are the same.

3.4 Expressing General Non-Recursive Multi-Case Types in the λ -calculus

It is now a straightforward step to come up with a solution for arbitrary non-recursive ADT's. Just combine the two solutions from above. Let us look at the definition of the function `warm` that takes a `Temperature` as an argument:

```
warm :: Temperature -> Boolean
warm (Fahrenheit f) = f > 90
warm (Celsius c)    = c > 30
```

We have to find encodings for `(Fahrenheit f)` and `(Celsius c)`. The enumeration example tells that we should make a λ -expression with 2 arguments that returns the first argument for `Fahrenheit` and the second argument for `Celsius`. The container solution (as used for `Tuple`) tells us that we should feed the argument of `Fahrenheit` or `Celsius` to a continuation function. Combining these two solutions we learn that `Fahrenheit` and `Celsius` should both have 3 arguments. The first one to be used for the closure and the second and third as continuation arguments. `Fahrenheit` should choose the first continuation argument and apply it to its first argument and `Celsius` should do the same with the second continuation argument:

$\text{Fahrenheit} \equiv \lambda t f c . f t$
 $\text{Celsius} \equiv \lambda t f c . c t$

Using this encoding the definition of `warm` becomes:

$\text{warm} \equiv \lambda t . t (\lambda f . f > 90) (\lambda c . c > 30)$

In the body the first argument of `t` represents the Fahrenheit case and the second one the Celsius case.

Also in this non-recursive case the Scott and Church approach do not differ.

3.5 Recursive Types in the λ -calculus: the Scott Encoding

In the Scott Encoding the previous strategy, as used for `Temperature`, is also applied to recursive types. As a matter of fact, the Scott Encoding ignores the fact that we deal with a recursive type! Let us look for example at `Nat` and `List`. Applying the strategy we used for `Temperature` for `Nat` we obtain the following definitions:

$\text{Zero} \equiv \lambda z s . z$
 $\text{Suc} \equiv \lambda n z s . s n$

Applying the same strategy for `List`, we obtain:

$\text{Nil} \equiv \lambda n c . n$
 $\text{Cons} \equiv \lambda x xs . n c . c x xs$

Functions like predecessor, head and tail can now easily be defined:

$\text{pred} \equiv \lambda n . n \text{undef} (\lambda m . m)$
 $\text{head} \equiv \lambda xs . xs \text{undef} (\lambda x xs . x)$
 $\text{tail} \equiv \lambda xs . xs \text{undef} (\lambda x xs . xs)$

Note that `pred` and `tail` have constant time complexity!

As another example we give the Scott Encoding of the `fold` functions for `Nat` and `List`. The Haskell definition `foldNat` is given by:

$\text{foldNat } f x \text{ Zero} = x$
 $\text{foldNat } f x (\text{Suc } n) = f (\text{foldNat } f x n)$

The conversion for the Scott encoding of `Nat` is straightforward, the bodies of the two cases simply appear as the first and second argument of `n` (later on we show how to remove the recursive call for `foldNat`):

$\text{foldNat} \equiv \lambda f x n . n x (\lambda n . f (\text{foldNat } f x n))$

For `foldList`, the Haskell definition is:

$\text{foldList } f d [] = d$
 $\text{foldList } f d (h:t) = f h (\text{foldList } f d t)$

Using the Scott encoding for lists this becomes:

$\text{foldList} \equiv \lambda f d xs . xs d (\lambda h t . f h (\text{foldList } f d t))$

The Scott encoding of ADT's is completely equivalent to their counterparts in Haskell and Clean. Functions acting on them can be straightforwardly converted to their Scott versions.

3.6 Recursive Types in the λ -calculus: the Church Encoding

Church uses an entirely different approach for the encoding of recursive data types.

The Church definitions of natural numbers are:

$$\begin{aligned} \text{Zero} &\equiv \lambda f x . x \\ \text{Suc} &\equiv \lambda n f x . f (n f x) \end{aligned}$$

If we compare this to the Scott approach we see that, instead of feeding only n to the continuation function f , the result of $n f x$ is fed to it. But this is exactly the same thing as what happens in the `fold` function. The definition of `foldNat` for Church encoded numerals can therefore be given by:

$$\text{foldNat} \equiv \lambda f x n . n f x$$

In [5] Hinze states that Church numerals are actually folds in disguise. As a consequence only primitive recursive functions on numbers can be easily expressed using the Church encoding. For functions that need general recursion (or functions for which the result for `suc n` cannot be expressed using the result for `n`) we run into troubles. Church himself was not able to solve this problem, but Kleene found a way out during a visit to the dentist (as described by Barendregt in [2]). A nice example of his solution is the predecessor function, which could be easily expressed using the Scott encoding, as we saw earlier. To define it using the Church encoding Kleene used a construction with pairs (`Tuple`):

$$\text{pred} \equiv \lambda n . \text{snd}(n (\lambda p . \text{Tuple} (\text{Suc} (\text{fst } p)) (\text{fst } p)) (\text{Tuple } \text{Zero } \text{Zero}))$$

Each pair combines the result of the recursive call with the previous element. A disadvantage of this solution, besides that it is hard to comprehend, is that it has complexity $O(n)$ while the Scott version has constant complexity.

The Church encoding for lists together with the function `tail` is given by:

$$\begin{aligned} \text{Nil} &\equiv \lambda f x . x \\ \text{Cons} &\equiv \lambda h t f x . f h (t f x) \end{aligned}$$

$$\text{tail} \equiv \lambda xs . \text{snd} (xs (\lambda x rs . \text{Tuple} (\text{Cons } x (\text{fst } rs)) (\text{fst } rs)) (\text{Tuple } \text{Nil } \text{Nil}))$$

Also here the definition of `Cons` behaves like a fold (a `foldr` actually). Again, we need the pair construction from Kleene for `tail`. The definition of `foldList` for Church encoded lists is given by:

$$\text{foldList} \equiv \lambda f d xs . xs f d$$

3.7 The Scott Encoding: the General Case

In general the mapping of an ADT to λ -expressions using the Scott encoding is defined as follows. Given an ADT definition in `Haskell` or `Clean`:

$$\text{data type_name } t_1 \dots t_k = C_1 t_{1,1} \dots t_{1,n_1} \mid \dots \mid C_m t_{m,1} \dots t_{m,n_m}$$

Then this type definition with m constructors can be mapped to m λ -expressions:

$$\begin{aligned}
C_1 &\equiv \lambda v_{1,1} \dots v_{1,n_1} f_1 \dots f_m . f_1 v_{1,1} \dots v_{1,n_1} \\
&\dots \\
C_m &\equiv \lambda v_{m,1} \dots v_{m,n_m} f_1 \dots f_m . f_m v_{m,1} \dots v_{m,n_m}
\end{aligned}$$

Consider the (multi-case) pattern-based function f in Haskell or Clean defined on this type:

$$\begin{aligned}
f (C_1 v_{1,1} \dots v_{1,n_1}) &= \text{body}_1 \\
&\dots \\
f (C_m v_{m,1} \dots v_{m,n_m}) &= \text{body}_m
\end{aligned}$$

This function is converted to the following λ -expression (of course, the bodies should also be encoded):

$$\begin{aligned}
f &\equiv \lambda x . x \\
&\quad (\lambda v_{1,1} \dots v_{1,n_1} . \text{body}_1) \\
&\quad \dots \\
&\quad (\lambda v_{m,1} \dots v_{m,n_m} . \text{body}_m)
\end{aligned}$$

3.8 From Church to Scott and back

It is straightforward to convert Church and Scott encoded numerals into each other. Because a fold replaces constructors by functions and Church numerals are actually folds, we can obtain the Scott representation by substituting back the Scott versions of the constructors:

$$\text{toScott} \equiv \lambda n . n \text{ Suc}_s \text{ Zero}_s$$

To go from Scott to Church we should use the Scott version of *foldNat*:

$$\text{toChurch} \equiv \lambda n f x . \text{foldNat } f x n$$

The conversions between the Church and Scott encoding for lists are given by:

$$\begin{aligned}
\text{toScottList} &\equiv \lambda xs . xs \text{ Cons}_s \text{ Nil}_s \\
\text{toChurchList} &\equiv \lambda xs f d . \text{foldList } f d xs
\end{aligned}$$

The list definitions are completely equivalent to those for numbers. They only use a different fold function in *toChurchList* and different constructors in *toScottList*. For other recursive ADT's similar transformations can be defined.

In the remainder of this paper we will concentrate on defining algorithms in the λ -calculus using the Scott encoding.

4 Defining Functions using the Scott Encoding

Now we know how to represent ADT's we can concentrate on functions. We already gave some examples of them above (*ifte*, *fst*, *snd*, *head*, *tail*, *pred*, *warm*, *foldNat*, *foldList*). The more interesting examples are the recursive functions. The standard technique for defining a recursive function in the λ -calculus is to use a fixed point operator. Let us look for example at addition for Peano numbers in Haskell:

`add Zero     m = m`
`add (Suc n) m = Suc (add n m)`

Using the Scott encoding, this becomes:

$add_0 \equiv \lambda n\ m . n\ m\ (\lambda n . Suc\ (add_0\ n\ m))$

We now have to get rid of the recursive macro add_0 in this definition. The standard way to do this is with the use of the Y fixed point combinator:

$add_Y \equiv Y\ (\lambda add\ n\ m . n\ m\ (\lambda n . Suc\ (add\ n\ m)))$
 $Y \equiv \lambda h . (\lambda x . h\ (x\ x))\ (\lambda x . h\ (x\ x))$

There is, however, another way to represent recursion. Instead of using a fixed point operator we can also give the recursive function itself as an argument (like this is done in the argument of Y in add_Y):

$add \equiv \lambda add\ n\ m . n\ m\ (\lambda n . Suc\ (add\ add\ n\ m))$

The price to pay is that each call of `add` should have `add` as an argument. The gain is that we do not need the fixed point operator any more. This definition is also more efficient, because it uses fewer reduction steps during reduction than the fixed-point version. The following example shows how add should be used to add one to one (note the double `add` in the call):

$(\lambda add . add\ add\ (Suc\ Zero)\ (Suc\ Zero))\ add$

4.1 Mutually Recursive functions

For mutually recursive functions, we have to add all mutually recursive functions as arguments for each function. An example to clarify this:

`isOdd Zero     = False`
`isOdd (Suc n) = isEven n`
`isEven Zero    = True`
`isEven (Suc n) = isOdd n`

This can be represented by λ -expressions as:

$isOdd \equiv \lambda isOdd\ isEven\ n . n\ False\ (\lambda n . isEven\ isOdd\ isEven\ n)$
 $isEven \equiv \lambda isOdd\ isEven\ n . n\ True\ (\lambda n . isOdd\ isOdd\ isEven\ n)$

5 Converting Algorithms to the λ -calculus

We now have all ingredients ready for converting complete programs. The last step to be made is combining everything into a single λ -expression. For example, if we take the `add 1 1` example from above, and substitute all macros, we obtain:

$(\lambda add . add\ add\ ((\lambda n\ z\ s . s\ n)(\lambda z\ s . z))\ ((\lambda n\ z\ s . s\ n)(\lambda z\ s . z)))$
 $(\lambda add\ n\ m . n\ m\ (\lambda n . (\lambda n\ z\ s . s\ n)\ (add\ add\ n\ m)))$

Using normal order (outermost) reduction this reduces to:


```
λz s. s (λz s. s (λz s. z))
```

which indeed represents the desired value 2. We can improve the readability by introducing explicit names for `zero` and `suc` by abstracting out their definitions:

```
(λzero suc .
  (λadd .
    add add (suc zero) (suc zero))
  (λadd n m . n m (λn . suc (add add n m)))
  (λz s.z) (λn z s.s n))
```

Here we applied a kind of inverted λ -lifting. We have used smart indentation to make the expression better readable. Note the nesting in this definition: the definition of `add` is inside the scope of the variables `suc` and `zero`, because its definition depends on their definitions. In this way the macro reference `Suc` in the definition of `add` can be replaced by a variable `suc`.

As another example, the right hand side of the Haskell function:

```
main = isOdd (Suc (Suc (Suc Zero)))
```

can be written as:

```
(λisOdd isEven . isOdd isOdd isEven (Suc (Suc (Suc Zero)))) isOdd isEven
```

and after substituting all macro definitions and abstracting out definitions:

```
(λtrue false zero suc .
  (λisOdd isEven .
    isOdd isOdd isEven (suc (suc (suc zero))))
  (λisOdd isEven n . n false (λn . isEven isOdd isEven n))
  (λisOdd isEven n . n true (λn . isOdd isOdd isEven n)))
  (λt f.t) (λt f.f) (λz s.z) (λn z s.s n))
```

Which reduces to:

```
λt f . t
```

Which shows that 3 is indeed an odd number.

5.1 Formalizing the Conversion

Above we mentioned the operation of abstracting out definitions. Here we make this more precise. The conversion of a program into a closed λ -expression proceeds in a number of steps:

1. Remove all syntactic sugar like `zf`-expressions, `where` and `let` expressions.
2. Eliminate algebraic data types by converting them to their Scott encoding.
3. Eliminate pattern-based function definitions by using the Scott encoding.
4. Remove (mutually) recursion by the introduction of extra variables.
5. Make a dependency sort of all functions, resulting in an ordered collection of sets. So the first set contains functions that do not depend on other functions (e.g. the Scott encoded ADT's). The second set contains functions that only depend on functions in the first set, etc. We can do this because all possible cycles are already removed in the previous step.

6. Construct the resulting λ -expression by nesting the definitions from the different dependency sets. The outermost expression consists of an application of a λ -expression with as variables the names of the functions from the first dependency set and as arguments the λ -definitions of these functions. The body of this expression is obtained by repeating this procedure for the remainder dependency sets. The innermost expression is the main expression.

The result of this process is:

```
(λfunction_names_first_set .
  (λfunction_names_second_set .
    ...
    (λfunction_names_last_set .
      main_expression)
    function_definitions_last_set)
  ...
  function_definitions_second_set)
function_definitions_first_set
```

6 Eratosthenes' Prime Sieve as a Single λ -expression

As a last, more convincing example, we convert the following Haskell version of the Eratosthenes prime sieve algorithm to a single λ -expression:

```
data Nat          = Zero | Suc Nat
data Inlist t     = Cons t (Inlist t)
nats n            = Cons n (nats (Suc n))
sieve (Cons Zero xs) = sieve xs
sieve (Cons (Suc k) xs) = Cons (Suc k) (sieve (rem k k xs))
rem p Zero (Cons x xs) = Cons Zero (rem p p xs)
rem p (Suc k) (Cons x xs) = Cons x (rem p k xs)
```

```
main = sieve (nats (Suc (Suc Zero)))
```

Here we use infinite lists for the storage of numbers and the resulting primes. `sieve` filters out the zero's in a list and calls `rem` to set multiples of prime numbers to zero. Applying the first four steps of the conversion procedure results in:

```
Zero  ≡ λz s . z
Suc   ≡ λn z s . s n
Cons  ≡ λx xs c . c x xs
nats  ≡ λnats n . Cons n (nats nats (Suc n))
sieve ≡ λsieve ls . ls (λx xs . x (sieve sieve xs)
                        (λk . Cons x (sieve sieve (rem rem k k xs))))
rem   ≡ λrem p k ls . ls (λx xs . k (Cons Zero (rem rem p p xs))
                        (λk . Cons x (rem rem p k xs)))
main ≡ sieve sieve (nats nats (Suc (Suc Zero)))
```

The dependency sort results in:

```
[{zero,suc,cons},{rem,nats},{sieve},{main}]
```

Putting everything into a single λ -expression this becomes:

```
(λzero suc cons .
  (λrem nats .
    (λsieve .
      sieve sieve (nats nats (suc (suc zero))))
    sieve)
  rem nats)
Zero Suc Cons
```

And after substituting the λ -definitions for all macros:

```
(λzero suc cons .
  (λrem nats .
    (λsieve .
      sieve sieve (nats nats (suc (suc zero))))
      (λsieve ls . ls (λx xs . x (sieve sieve xs)
        (λk . cons x (sieve sieve (rem rem k k xs))))))
      (λrem p k ls . ls (λx xs . k (cons zero (rem rem p p xs))
        (λk . cons x (rem rem p k xs))))
      (λnats n . cons n (nats nats (suc n))))
      (λz s . z) (λn z s . s n) (λx xs c . c x xs))
```

Which reduces to an infinite λ -expression starting with:

```
λc. c (λz s. s (λz s. s (λz s. z))) (λc. c (λz s. s (λz s. s (λz s. s (λz s. z))))
(λc. c (λz s. s (λz s. s (λz s. s (λz s. s (λz s. s (λz s. s (λz s. z)))))) ...
```

One can recognize the start of a list containing: 2, 3 and 5. Using single character names the expression reduces to a 143 character length definition:

```
(λzsc.(λrf.(λe.ee(ff(s(sz)))))(λel.lλht.h(eet)λk.ch(ee(rrkkt))))
(λrpkl.lλht.k(cz(rrppt))λk.ch(rrpkt))(λfn.cn(ff(sn)))(λzs.z)(λnzs.zn)(λhtc.cht)
```

This λ -term can also be considered as a constructive definition of what prime numbers are. An even shorter definition of a prime number generator in the λ -calculus can be found in Tromp [11].

7 Discussion

We already indicated that the Scott encoding just combines the techniques used for encoding booleans and tuples in the Church encoding as described in standard λ -calculus text books and courses. The Scott and Church encodings only differ for recursive types. A Church encoded type just defines how functions should be folded over an element of the type. A fold can be characterized as a function that replaces constructors by functions. The Scott encoding just packages information into a closure. Recursiveness of the type is not visible at this level. Of course, this is also the case for ADT's in functional languages, where recursiveness is only visible at the type level and not at the element level.

The representation achieved using the Scott encoding is equivalent to that of ADT definitions in modern functional programming languages and allows for an

similar realization of functions defined on ADT's. Also the complexity (efficiency) of these functions is similar to their equivalents in functional programming languages. This in contrast to their counterparts using the Church encoding that sometimes have a much worse complexity. Therefore, from a programmers perspective the Scott encoding is better than the Church encoding.

An interesting question now is: Why is the Scott encoding relatively unknown and almost never mentioned in textbooks on the λ -calculus? The encoding is simpler than the Church encoding and allows for a straightforward implementation of functions acting on data types. Of course, the way ADT's are represented in modern functional programming languages is rather new and dates from languages like ISWIM [7], HOPE [3] and SASL [13, 12] and this was long after the Church numerals were invented. Furthermore, ADT's are needed and defined by programmers, who needed an efficient way to define new types, which is rather irrelevant for mathematicians and logicians studying the λ -calculus.

In [6] it is shown that this representation of functional programs can be used to construct very efficient, simple and small interpreters for lazy functional programming languages. These interpreters only have to implement β -reduction and no constructors nor pattern matching.

Altogether, we argue that the Scott encoding also should have its place in λ -calculus textbooks and courses and in λ -calculus courses for computer scientist this encoding should have preference over the Church encoding.

8 Conclusions

In this paper we showed how the λ -calculus can be used to express algorithms and Algebraic Data Types in a way that is close to the way this is done in modern functional programming languages. To achieve this, we used a rather unfamiliar encoding of ADT's attributed to Scott. We showed that this encoding can be considered as a logical combination of the way how enumerations (like booleans) and containers (like tuples) are normally encoded in the λ -calculus. The encoding differs from the Church encoding and the connecting element between them is the fold function.

For recursive functions we did not use the standard fixed-point combinators, but instead used a simple technique where an expression representing a recursive function is given (a reference to) itself as an argument. In this way the recursion is made more explicit and this also results in a more efficient implementation using fewer reduction steps.

We also sketched a systematic method for converting Haskell or Clean like programs to closed λ -expressions.

Altogether we have shown that it is possible to express a functional program in a concise way as a λ -expression and demonstrated that the λ -calculus is indeed a universal programming language in a convincing way.

References

1. H. Barendregt. *The lambda calculus, its syntax and semantics (revised edition)*, volume 103 of *Studies in Logic*. North-Holland, 1984.
2. H. Barendregt. The impact of the lambda calculus in logic and computer science. *The Bulletin of Symbolic Logic*, 3(2):181–215, 1997.
3. R. M. Burstall, D. B. MacQueen, and D. T. Sannella. Hope: An experimental applicative language, 1980.
4. H. Curry, J. Hindley, and J. Seldin. *Combinatory Logic*, volume 2. North-Holland Publishing Company, 1972.
5. R. Hinze. Theoretical pearl Church numerals, twice! *Journal of Functional Programming*, 15(1):1–13, 2005.
6. J. Jansen, P. Koopman, and R. Plasmeijer. Efficient interpretation by transforming data types and patterns to functions. In H. Nilsson, editor, *Revised Selected Papers of the 7th Trends in Functional Programming '06*, volume 7, pages 73–90, Nottingham, UK, 2006. Intellect Books.
7. P. J. Landin. The next 700 programming languages. *Commun. ACM*, 9(3):157–166, 1966.
8. T. A. Mogensen. Efficient Self-Interpretation in Lambda Calculus. *Journal of Functional Programming*, 2:345–364, 1994.
9. J. Steensgaard-Madsen. Typed representation of Objects by Functions. *ACM Transactions on Programming Languages and Systems*, 11(1):67–89, jan 1989.
10. A. Stump. Directly reflective meta-programming. *Journal of Higher Order and Symbolic Computation*, 2008.
11. J. Tromp. John’s lambda calculus and combinatory logic playground, 2012. <http://homepages.cwi.nl/~tromp/cl/cl.html>.
12. D. Turner. Some History of Functional Programming Languages, 2012. Invited talk, Trends in Functional Programming 2012, St. Andrews, United Kingdom, TFP 2012.
13. D. A. Turner. A new implementation technique for applicative languages. *Softw., Pract. Exper.*, 9(1):31–49, 1979.